

Analysis: Fence Construction

For both solutions discussed below, we model the situation as a [planar graph](#) G with endpoints as nodes and fences as edges. Getting a representation of this graph as an adjacency list takes quasilinear time on the input (because we need to identify endpoints, which may add a logarithmic factor or an expected constant factor depending on whether we use trees, hashing or sorting to do it).

Test set 1

Let us assume for a brief moment that there is no prescribed partial ordering (for instance, when $K = 1$). There are many greedy algorithms that would retrieve a correct ordering of the fences. You can try working the fences from the outside in or from the inside out, cleverly using [convex-hull](#)-type algorithms. Each way has its own pros and cons, but many of them also have the property that the reverse of the ordering they produce is also a valid ordering. This is enough to deal with K up to 2: retrieve a greedy order. If the partial order is obeyed, return that; otherwise, return the reverse. One of the two is guaranteed to work. Another way that works for some greedy alternatives is that they may allow us to fix a single fence as the first one to be built and then work from there. If you fix fence 1 to be the first one built, the partial order is guaranteed to be respected.

We will describe a different option than the one discussed above, because it is simpler to describe, prove and implement. Build any [spanning tree](#) T of G that contains fence 1. Build the fences in T first in any order that starts with fence 1. This guarantees that we respect the partial order. Since we are building fences that close no cycle, there are no restrictions to moving the printer so far: any point not on a fence is reachable.

For each the remaining fences f , consider the cycle that is formed when f is added to T . That cycle represents a [simple polygon](#). If we simply build these remaining fences in increasing order of area of that polygon (breaking ties arbitrarily) we obtain a valid order. The reason is that we can always keep the printer on the "outside" of all closed cycles. Since the next cycle to be closed at any time has area no less than any already closed cycles, it cannot be contained in any of them, which means the fence that is closing it is also on the "outside", and so the printer can get arbitrarily close to the fence's position and find a point from where the fence can be built.

Notice that this algorithm can be implemented in quadratic time. Building a spanning tree takes linear time. For each fence outside of the spanning tree, we take linear time to get the area of the polygon, which leads to quadratic overall. Then, we only need to sort the remaining fences, which can be done in faster-than-quadratic time.

Test set 2

For Test set 2, a bit of theoretical knowledge is required. Build the [dual](#) graph of G and call it H . Moreover, let $G(S)$ and $H(S)$ be the graph corresponding to a given set of fences S and its dual, respectively. As usual, when we name faces or nodes of H , we include the face that is on the outside (i.e., the only one with infinite area).

When we finish our work, the printer will be left in one of the faces of G , that is, one of the nodes of H , and the last fence we built will be one of the edges that are adjacent to that face. If we step through the build order backwards, right before building the last fence f , the printer is on a face of $G(F - \{f\})$, where F is the set of all the fences. For any set S and any fence f , there is a strong

relationship between $G(S)$ and $G(S - \{f\})$, and correspondingly, $H(S)$ and $H(S - \{f\})$. $G(S - \{f\})$ is the result of removing f , and possibly its endpoints if they become isolated, from $G(S)$. Correspondingly, $H(S - \{f\})$ is either equal to $H(S)$ or the result of merging two nodes of $H(S)$ into one: f may have a single adjacent face in $G(S)$ or two, which leads to nothing changing or two nodes merging in $H(S)$, respectively.

Focusing back on $G(F - \{f\})$, we know that the next-to-last fence g to be built has to be one of the fences that is adjacent to a face adjacent to f . The one right before that is adjacent to a face adjacent to either f or g , and so on. That means any ordering of the fences is a possible search on the graph X of fences as nodes where two fences are adjacent in X if they are adjacent to the same face of G . A search on a graph is an ordering of the nodes such that any prefix of the ordering is connected. Breadth-first-searches and depth-first-searches are examples of searches, but not the only ones.

Now that we have identified valid orders with searches on a graph, we can greedily build one that respects the partial order. From this point on, we will build the reverse of the output. Let us fix the first fence of our order f . From here, we can keep track of the reached fences. Any reached fence that is not in the partial order can be added immediately to our order, as it only adds more reachable fences and it cannot violate the partial order. Fences that are in the partial order, however, need to be added respecting that, which we can also do greedily. If at some point all the reachable fences are in the partial order but neither is the next one, we have failed and we need to choose a different f .

The explanations above lead to a quadratic algorithm: building G and then H takes quasilinear time. Building X from H takes quadratic time (since it may have a quadratic number of edges). The greedy algorithm takes constant time for each fence it adds to the order, and we may need to try a linear number of starting fences f , so that time is quadratic overall.

The algorithm can actually be improved to quasilinear overall: there is no need to build X explicitly. We can use X implicitly from H and only inspect adjacencies to fences that haven't been reached. That removes the quadratic cost of explicitly building X . As for trying every f , we can prove that fixing f to be the first fence in the partial order (that is, fence **K**, since we are building a reverse order) is always optimal. For a valid search of X that respects the partial order, reversing the prefix that ends on fence **K** is also a valid search that respects the partial order, and has fence **K** first.

Analysis: Field Trip

Test Set 1 (Visible)

Since the number of kids and the grid dimensions are small in Test set 1, it is sufficient to develop a correct greedy heuristic for where to move the teacher and then run the simulation to completion.

The first observation is that the problem can be solved independently in each dimension (x and y). At each time step, kid i always moves exactly 1 step closer to person $i-1$ in both x and y until both people occupy the same position in that dimension. Diagonal movement happens when the two people have different positions in both dimensions; orthogonal movement happens when the two people have the same position in one dimension but different positions in the other dimension; and no movement happens when the people share the same position in both dimensions.

The second observation is that when we take the one-dimensional perspective, the time is limited by only the most distant kids from the teacher in each dimension and direction. Let K be the number of the most distant kid in a certain dimension and direction; if multiple kids share that same position, pick the one with the lowest number. By assumption, all kids with a number lower than K are either closer to or on the opposite side of the teacher than kid K , including kid $K-1$. Therefore, kid K will move one step in the direction of the teacher. The effect inducts to kids with numbers higher than K ; now that kid K has moved closer to the teacher, the first kid with number greater than K that has a distance to the teacher equal to kid K 's original distance becomes the most distant kid and is guaranteed to step toward the teacher.

Hence, the correct heuristic is to minimize the distance between the teacher and the most distant kid, treating each dimension independently. To do this, calculate the most distant kid in each dimension, and move the teacher toward each of those kids in their respective dimension. If two kids are equally distant but in opposite directions, do not move the teacher.

An equivalent statement of the heuristic is that the teacher always moves toward the average of the minimum and maximum coordinates in each dimension, not moving if the teacher is already at that center point. If the maximum difference in a certain dimension is odd, the teacher can oscillate between the two central spaces in that dimension; this does not affect the correctness of the heuristic.

A few *incorrect* greedy heuristics include:

1. Move the teacher toward the single most distant kid. This is wrong because x and y need to be independent.
2. Move the teacher toward the average or median kid position. This is wrong because it biases the teacher toward a cluster of kids instead of dealing with outliers.
3. Move the teacher toward first kid that is not on their current space. This is wrong because the teacher could waste time making a cluster smaller before moving toward outliers.

Test Set 2 (Hidden)

Running the simulation to completion clearly does not work with the higher bounds.

By the greedy heuristic described above, at each time step, since the most distant student in each direction and dimension moves one step closer to the teacher, in each dimension, the minimum coordinate increases by 1 and the maximum coordinate decreases by 1. The problem

in each dimension is solved once the minimum and maximum coordinates equal each other. Therefore, we can calculate the total time by taking the difference between the minimum and maximum coordinates in each direction and dividing by 2 (rounding up), choosing the larger of the value in the x dimension and the value in the y dimension. This requires looking at each kid once and therefore takes $O(N)$ time.

Analysis: Name-Preserving Network

How hard can this be?

At first glance, test set 1 may seem to be approachable even by hand, since only one design is required, and we have considerable flexibility in the number of computers to use. But it turns out to be difficult to satisfy all of the following at once:

- Each computer must use exactly 4 links.
- The network must be connected.
- The computers must be uniquely distinguishable even after their IDs are permuted, and we must know how to distinguish them.

For example, we might try creating a ring of computers in which each one is linked to its two neighbors and neighbors' neighbors. This satisfies the 4-link and connected conditions. But this design is highly symmetric in a way that makes the computers less distinguishable. We can try introducing some irregularities (by swapping some edges around; more on this later) to disrupt the symmetry, but then it becomes even harder to eyeball the design — and how can we "find" our computer again after the judge has permuted their identities?

Harder than it looks

We need a way to give each computer a unique label. Labels need to be such that:

- They are id-agnostic — that is, the label of a computer depends only on the link topology, and not on its own or other computers' ids.
- There exist networks such that each computer has a unique label, and moreover, we can somewhat efficiently find those networks.
- We can efficiently compute the labels of a given network.

If we have a labeling scheme that has all these properties, the solution writes itself: find a network of appropriate size, compute the labels on it and the labels on the shuffled network given by the judge, and match the computers by label.

There are many possible labeling schemes, and experimentation can go a long way to find them. We describe two below, one for the theoretically inclined, and one more manual. There also exist ways to explicitly construct networks that guarantee a particular labeling scheme. Both the constructions and labeling schemes are somewhat complicated and require casework, and only those constructions with labeling schemes that are quick to compute will work for this problem. Constructions are of course significantly easier for Test set 1, as we only need to build one network and we can use whichever network size (between 10 and 50) we want.

Let us start with the theoretical one, since it's shorter to write. We can model the network as an undirected graph in which each node has degree 4. If we take the [adjacency matrix](#) of the graph and compute its p -th power, we obtain a matrix in which cell (i,j) gives the number of paths of length p between nodes i and j . In the context of this problem, we do not particularly care about these paths per se, but we do care that the count of paths for a pair of nodes (i,j) depends on the entire topology of the network. So, if we are lucky enough, the set of values $(i,1), (i,2), \dots$ will be unique for each i . As it turns out, most graphs of a given size yield unique levels for $p = 7$, and quite possibly for other values as well. We generated graphs for every size between 10 and 50 and only needed to try a second time twice, and never a third time.

For a more manual approach, we can start by noticing that all computers look similar if we focus on them in isolation; they all use four outgoing links. The same is true of each of a computer's four neighbors. But what if we look at how the neighbors are connected to each other? Specifically, let us label a computer X via a multiset of four values Y_i , one for each computer linked to X , where Y_i is the number of other computers that are neighbors of both X and Y_i . For example, suppose that computer 1 is directly linked to computers 2, 3, 4, and 5, and that there are additional direct links 2-3, 2-4, and 4-5 (and all other links from 2, 3, 4, and 5 are to other computers). Then our multiset for computer 1 would be $\{1, 1, 2, 2\}$.

This labeling will not get us far enough in distinguishing computers, since there are not many possible labels of the form described above. But we can take this approach deeper! Let us label each computer as described above, and call those the level 1 labels. Then, we give each computer a level 2 label that is defined as the multiset of the level 1 labels of its neighbors — that is, each level 2 label is a multiset of four multisets. We can even go on to add a level 3 label that is the multiset of the neighbors' level 2 labels, and so on. It turns out that level 4 labels give us enough power to tell computers apart, at least within the 10 to 50 computer range. Moreover, since labeling a computer only requires looking at a computer's four neighbors and a finite number of possible connections between them, one round of the labeling process takes $O(\text{computers})$ time, as does the full process of getting the level 4 labels. This is more efficient than other possible approaches that, e.g., label a computer by the smallest or largest cycle it is part of.

Armed with either labeling method, we just need a way to create network designs to try them on. Generating edges at random has a really low probability of making all degrees exactly 4, so we need something better. One simple possibility is to shuffle a list of exactly 4 times each id, and link the first two ids, the third and fourth, and so on. If this creates a self-loop or a repeated link, repeat until it doesn't. The probability of self-loops or repeated edges is small enough for this to finish quickly for every size.

Another way to generate graphs that is less chaotic is to start with the ring design mentioned above, and then repeatedly mutate it as follows:

1. Randomly select a computer A and one of its neighbors B .
2. Randomly select a neighbor C of A and a neighbor D of B , such that C is not already a neighbor of B , and D is not already a neighbor of A .
3. Delete the links $A-C$ and $B-D$, and add the links $A-D$ and $B-C$.

This mutation method guarantees that each computer always has four links, and that the network remains connected. (Since we had $C-A-B-D$ before and we have $C-B-A-D$ after, any two computers that were directly or indirectly connected before are still connected.)

Most networks generated in this way (after, e.g., 1000 mutations) turn out to have unique labels on the computers. When we get one that does not (either because the network has some inherent symmetry, or because our labeling method isn't powerful enough for the network), we can just throw it away and try again. Our strategy should take at most a few seconds to discover usable network designs for all possible test cases, and then the rest is just implementation of interactions with the judge.

Finally, notice that, if necessary, graphs for all necessary sizes could be pre-computed offline and hardcoded into the solution. As long as the labeling method is efficient, it is OK for the process of finding the graphs to be a bit slow, as long as it finishes within the contest time! Nevertheless, all ideas presented above are fast enough to require no pre-computation.

Analysis: Raise the Roof

Test set 1

In Test set 1, the number of columns is small enough to allow a brute force solution. The problem guarantees that there is at least one correct order, so we can try all possible orders (there are at most $10! = 3628800$) until we find a correct one. To check a given order, we check every prefix of length at least 3, build the roof implied by the last 3 columns of the prefix, and then check that all other columns are below it. Since the statement guarantees no 3 points are collinear, any subset of 3 points determines a single plane. There are several ways to check if a roof is valid; here is one that involves only integer arithmetic:

Let $p_1, p_2, \dots, p_k$ be the k points in the prefix. Let q be the plane that contains p_{k-2}, p_{k-1} and p_k . For each i between 1 and $k - 3$, inclusive, we need to check whether p_i is above q . Notice that we can subtract p_k from all points and the answer to the question doesn't change. Thus, we further assume $p_k = (0, 0, 0)$. Let v be a [normal](#) to the plane q . Project p_i onto v via [dot product](#) and check whether v and the projection are on the same side of q . If they are, then p_i is above q , otherwise, it is not.

Since checking a prefix takes $O(N)$ time, and there are $O(N)$ prefixes to check for each of the $N!$ orders, this yields an $O(N! \times N^2)$ algorithm that may or may not be fast enough depending on your language and implementation. We can speed it up in several ways:

- Generate permutations by adding columns one at a time, and check each prefix as soon as it is generated. This reduces the complexity to $O(N! \times N)$ because each prefix is not re-checked for each permutation that starts with it. Additionally, when we find a prefix that does not work, we do not waste time checking other permutations beginning with that prefix, which can further reduce the search space in practice.
- Try a dynamic-programming-on-subsets approach to change the complexity to something like $O(2^N \times N^4)$, but whether or not that makes the solution faster in practice depends heavily on implementation details.

Test set 2

For Test set 2, any solution that is exponential or factorial in N is doomed from the start, so we need a different strategy. One possibility is to approach the problem in reverse: start with all columns, find a roof that works, remove a column, and repeat.

To formalize this a bit, we start by fixing the last two points q and p from the set of column tips. to be the last two columns, in that order. We discuss how to do this later. Then, we define s as the set of all column tips except p and q , and repeat the following until s is empty:

1. Find a point r such that the plane that contains p, q and r is above all current points.
2. Remove r from the set of current points and set $p = q$ and $q = r$.

The final output is the reverse of the order the points were taken from s , with p and q at the end.

To do the first step above (find r), we could just try every possible point in s and use the process described above to check the condition. This would yield an $O(N^2)$ algorithm for the step. To improve upon that, picture the plane containing the last roof, but now being supported by the remaining 2 fixed columns. For the very first iteration, the "last roof" needs to exist for this

algorithm to be valid, and how to define it depends on how we choose p and q . For the choice of p and q we explain below, the existence of a possible "last roof" is straightforward. We can rotate the plane around the axis defined by the segment connecting the tips of those two columns, and we want to find one of the columns that it touches first. (There can be up to two such columns, since we can rotate the plane in either direction.) In other words, we should choose a column tip r that minimizes the angle of rotation. That is equivalent to choosing the r that maximizes the angle between a normal to the plane that contains p , q and r and the normal to the "last roof". This yields a linear time algorithm for this step.

To choose the initial p and q , we can try every possibility, which adds an N^2 factor to the overall time complexity of the algorithm, or do something better. The last column with tip p can always be chosen as one of the tallest columns. Imagine a plane parallel to $z = 0$ that passes through p . The plane clearly passes above or touches all other columns, but it's not "pierced" by any column, so we can move it around to create a suitable roof. Then, we can choose q as a column such that the segment pq has minimal angle with the plane. This is similar to what we did above, except we are trying rotating the plane around a single point p until it touches another column q . This can be done by using the same method of comparing angles against a plane we described in the previous paragraph. This involves a one-time linear cost to find q , but then we only do the iteration for a single pair. Notice that, for this choice of p and q , there is a clear choice for the "last roof", which is the imaginary plane that we rotated to find q .

The optimal implementation of the ideas above yields a quadratic algorithm, which is the intended solution. Depending on which of the optimizations you skip, you can end up with a complexity between $O(N^2)$ and $O(N^5)$, which, together with the constant factors of your implementation and choice of language, determines whether your solution passes both sets, just Test set 1, or neither set.